Probabilistic programming language

- Wikipedia “A probabilistic programming language (PPL) is a programming language designed to describe probabilistic models and then perform inference in those models”

Probabilistic programming language

- Wikipedia “A probabilistic programming language (PPL) is a programming language designed to describe probabilistic models and then perform inference in those models”
- To make probabilistic programming useful
  - inference has to be as automatic as possible
  - diagnostics for telling if the automatic inference doesn’t work
  - easy workflow (to reduce manual work)
  - fast enough (manual work replaced with automation)

Probabilistic programming

- Enables agile workflow for developing probabilistic models
  - language
  - automated inference
  - diagnostics
- Many frameworks Stan, PyMC, Pyro (Uber), TFP (Google), Turing.jl, JAGS, ELFI, ...
  - Short review of the landscape:
    Štrumbelj et al. (2023). Past, Present, and Future of Software for Bayesian Inference. *Statistical Science*, 39(1):46-61. https://doi.org/10.1214/23-STS907.

Stan - probabilistic programming framework

- Language, inference engine, user interfaces, documentation, case studies, diagnostics, packages, ...
  - autodiff to compute gradients of the log density

mc-stan.org

Stan - probabilistic programming framework

- Language, inference engine, user interfaces, documentation, case studies, diagnostics, packages, ...
  - autodiff to compute gradients of the log density
- Most popular, 200K+ users in social, biological, and physical sciences, medicine, engineering, and business

mc-stan.org

Stan - probabilistic programming framework

- Language, inference engine, user interfaces, documentation, case studies, diagnostics, packages, ...
  - autodiff to compute gradients of the log density
- Most popular, 200K+ users in social, biological, and physical sciences, medicine, engineering, and business
- Several full time developers, 40+ developers, 100+ contributors

mc-stan.org

Stan - probabilistic programming framework

- Language, inference engine, user interfaces, documentation, case studies, diagnostics, packages, ...
  - autodiff to compute gradients of the log density
- Most popular, 200K+ users in social, biological, and physical sciences, medicine, engineering, and business
- Several full time developers, 40+ developers, 100+ contributors
- R, Python, Julia, Scala, Stata, command line interfaces
- 300+ R packages using Stan

mc-stan.org

Stan

- Stanislaw Ulam (1909-1984)
  - Monte Carlo method
  - H-Bomb

Binomial model - Stan code

Domain-specific language for constructing models
with common *distributed as* ~ notation

```
data {
  int<lower=0> N;          // number of experiments
  int<lower=0,upper=N> y; // number of successes
}

parameters {
  real<lower=0,upper=1> theta; // parameter of the binomial
}

model {
  theta ~ beta(1, 1);       // prior
  y ~ binomial(N, theta);  // observation / data model
}
```

Binomial model - Stan code

Domain-specific language for constructing models
with common *distributed as* ~ notation

```
data {
  int<lower=0> N;          // number of experiments
  int<lower=0,upper=N> y; // number of successes
}

parameters {
  real<lower=0,upper=1> theta; // parameter of the binomial
}

model {
  theta ~ beta(1, 1);       // prior
  y ~ binomial(N, theta);  // observation / data model
}
```

Binomial model - Stan code

Domain-specific language for constructing models
with common *distributed as* ~ notation

```
data {
  int<lower=0> N;          // number of experiments
  int<lower=0,upper=N> y; // number of successes
}

parameters {
  real<lower=0,upper=1> theta; // parameter of the binomial
}

model {
  theta ~ beta(1, 1);       // prior
  y ~ binomial(N, theta);  // observation / data model
}
```

Binomial model - Stan code

```
data {
  int<lower=0> N;          // number of experiments
  int<lower=0,upper=N> y; // number of successes
}
```

- Data type and size are declared
- Stan checks that given data matches type and constraints

Binomial model - Stan code

```
data {
  int<lower=0> N;          // number of experiments
  int<lower=0,upper=N> y; // number of successes
}
```

- Data type and size are declared
- Stan checks that given data matches type and constraints
  - If you are not used to strong typing, this may feel annoying, but it will reduce the probability of coding errors, which will reduce probability of data analysis errors

Binomial model - Stan code

```
parameters {
  real<lower=0,upper=1> theta; // parameter of the binomial
}
```

- Only continuous parameters allowed (discrete parameters can often be integerated out in the model block)
- Parameters may have constraints
- Stan makes transformation to unconstrained space and samples in unconstrained space
  - e.g. log transformation for `<lower=a>`
  - e.g. logit transformation for `<lower=a,upper=b>`

Binomial model - Stan code

```
parameters {
  real<lower=0,upper=1> theta; // parameter of the binomial
}
```

- Only continuous parameters allowed (discrete parameters can often be integerated out in the model block)
- Parameters may have constraints
- Stan makes transformation to unconstrained space and samples in unconstrained space
  - e.g. log transformation for `<lower=a>`
  - e.g. logit transformation for `<lower=a,upper=b>`
- For these declared transformation Stan automatically takes into account the Jacobian of the transformation (see BDA3 p. 21)

Binomial model - Stan code

```
model {
  theta ~ beta(1, 1);      // prior
  y ~ binomial(N, theta); // observation model
}
```

- `~` defines a *distribution statement*
  e.g. *y is distributed as* binomial(*N, theta*)

Binomial model - Stan code

```
model {
  theta ~ beta(1, 1);      // prior
  y ~ binomial(N, theta); // observation model
}
```

- ~ defines a *distribution statement*
  e.g. *y is distributed as* binomial(*N, theta*)
- these can be written also as *log density increment statements*
  left side of | denotes what is distributed as, e.g., binomial

```
model {
  target += beta_lpdf(theta | 1, 1);
  target += binomial_lpmf(y | N, theta);
}
```

Binomial model - Stan code

```
model {
  theta ~ beta(1, 1);      // prior
  y ~ binomial(N, theta); // observation model
}
```

- ~ defines a *distribution statement*
  e.g. *y is distributed as* binomial(*N, theta*)
- these can be written also as *log density increment statements*
  left side of | denotes what is distributed as, e.g., binomial

```
model {
  target += beta_lpdf(theta | 1, 1);
  target += binomial_lpmf(y | N, theta);
}
```

- `target` is the log posterior density (Lecture 4 discussed log)

Binomial model - Stan code

```
model {
  theta ~ beta(1, 1);      // prior
  y ~ binomial(N, theta); // observation model
}
```

- ~ defines a *distribution statement*
  e.g. *y is distributed as* binomial(*N, theta*)
- these can be written also as *log density increment statements*
  left side of | denotes what is distributed as, e.g., binomial

```
model {
  target += beta_lpdf(theta | 1, 1);
  target += binomial_lpmf(y | N, theta);
}
```

- `target` is the log posterior density (Lecture 4 discussed log)
- `_lpdf` for continuous, `_lpmf` for discrete distributions (left of |)

Binomial model - Stan code

```
model {
  theta ~ beta(1, 1);      // prior
  y ~ binomial(N, theta); // observation model
}
```

- ~ defines a *distribution statement*
  e.g. *y is distributed as* binomial(*N, theta*)
- these can be written also as *log density increment statements*
  left side of | denotes what is distributed as, e.g., binomial

```
model {
  target += beta_lpdf(theta | 1, 1);
  target += binomial_lpmf(y | N, theta);
}
```

- `target` is the log posterior density (Lecture 4 discussed log)
- `_lpdf` for continuous, `_lpmf` for discrete distributions (left of |)
- if `y` are data, and `theta` is a parameter, then that term defines log likelihood

Binomial model - Stan code

```
model {
  theta ~ beta(1, 1);      // prior
  y ~ binomial(N, theta); // observation model
}
```

- ~ defines a *distribution statement*
  e.g. *y is distributed as* binomial(*N, theta*)
- these can be written also as *log density increment statements*
  left side of | denotes what is distributed as, e.g., binomial

```
model {
  target += beta_lpdf(theta | 1, 1);
  target += binomial_lpmf(y | N, theta);
}
```

- `target` is the log posterior density (Lecture 4 discussed log)
- `_lpdf` for continuous, `_lpmf` for discrete distributions (left of |)
- if `y` are data, and `theta` is a parameter, then that term defines log likelihood
- for Stan sampler there is no difference between prior and likelihood, all that matters is the final `target`

Stan

- You can write in Stan language any program to compute the log density (Stan language is Turing complete)

Stan

- You can write in Stan language any program to compute the log density (Stan language is Turing complete)
- Stan compiles (transpiles) the model written in Stan language to C++
  - this makes the sampling for complex models and bigger data faster
  - also makes Stan models easily portable, you can use your own favorite interface and scripting language for manipulating data and inference results (e.g. R, Python, Julia, Stata, ...)

CmdStanR

CmdStanR is an R interface for Stan

```r
# Load CmdStanR
library(cmdstanr)

# Compile Stan model
mod_bin <- cmdstan_model(stan_file = 'binom.stan')

# Sample from the posterior given the model and data
d_bin <- list(N = 10, y = 7)
fit_bin <- mod_bin$sample(data = d_bin)

# Show summary and access draws
fit_bin$summary()
draws <- fit_bin$draws(format = "df")
```

CmdStanR

CmdStanR is an R interface for Stan

```r
# Load CmdStanR
library(cmdstanr)

# Compile Stan model
mod_bin <- cmdstan_model(stan_file = 'binom.stan')

# Sample from the posterior given the model and data
d_bin <- list(N = 10, y = 7)
fit_bin <- mod_bin$sample(data = d_bin)

# Show summary and access draws
fit_bin$summary()
draws <- fit_bin$draws(format = "df")
```

CmdStanR

CmdStanR is an R interface for Stan

```r
# Load CmdStanR
library(cmdstanr)

# Compile Stan model
mod_bin <- cmdstan_model(stan_file = 'binom.stan')

# Sample from the posterior given the model and data
d_bin <- list(N = 10, y = 7)
fit_bin <- mod_bin$sample(data = d_bin)

# Show summary and access draws
fit_bin$summary()
draws <- fit_bin$draws(format = "df")
```

CmdStanR

CmdStanR is an R interface for Stan

```r
# Load CmdStanR
library(cmdstanr)

# Compile Stan model
mod_bin <- cmdstan_model(stan_file = 'binom.stan')

# Sample from the posterior given the model and data
d_bin <- list(N = 10, y = 7)
fit_bin <- mod_bin$sample(data = d_bin)

# Show summary and access draws
fit_bin$summary()
draws <- fit_bin$draws(format = "df")
```

Stan

- Compilation (unless previously compiled model available)
- Pick random initial values for MCMC chains
- Run warm-up iterations including adaptation of mass matrix and step-size
- Sampling
- Generated quantities
- Save posterior draws
- Report divergences, ESS, $\widehat{R}$

Vaccine efficacy

- Randomized trial for influenza vaccine
- Participants were randomly assigned to vaccine and placebo groups:
  - out of 338 patients receiving the placebo, 16 were infected
  - out of 867 receiving the vaccine, 19 were infected

Vaccine efficacy

```
data {
  int<lower=0> N1;
  int<lower=0> y1;
  int<lower=0> N2;
  int<lower=0> y2;
}
parameters {
  real<lower=0,upper=1> theta1;
  real<lower=0,upper=1> theta2;
}
model {
  theta1 ~ beta(1, 1);
  theta2 ~ beta(1, 1);
  y1 ~ binomial(N1, theta1);
  y2 ~ binomial(N2, theta2);
}
generated quantities {
  real ve;
  ve = 1 - theta2 / theta1;
}
```

Vaccine efficacy

```
data {
  int<lower=0> N1;
  int<lower=0> y1;
  int<lower=0> N2;
  int<lower=0> y2;
}
parameters {
  real<lower=0,upper=1> theta1;
  real<lower=0,upper=1> theta2;
}
model {
  theta1 ~ beta(1, 1);
  theta2 ~ beta(1, 1);
  y1 ~ binomial(N1, theta1);
  y2 ~ binomial(N2, theta2);
}
generated quantities {
  real ve;
  ve = 1 - theta2 / theta1;
}
```

Vaccine efficacy

```
data {
  int<lower=0> N1;
  int<lower=0> y1;
  int<lower=0> N2;
  int<lower=0> y2;
}
parameters {
  real<lower=0,upper=1> theta1;
  real<lower=0,upper=1> theta2;
}
model {
  theta1 ~ beta(1, 1);
  theta2 ~ beta(1, 1);
  y1 ~ binomial(N1, theta1);
  y2 ~ binomial(N2, theta2);
}
generated quantities {
  real ve;
  ve = 1 - theta2 / theta1;
}
```

Vaccine efficacy

```
data {
  int<lower=0> N1;
  int<lower=0> y1;
  int<lower=0> N2;
  int<lower=0> y2;
}
parameters {
  real<lower=0,upper=1> theta1;
  real<lower=0,upper=1> theta2;
}
model {
  theta1 ~ beta(1, 1);
  theta2 ~ beta(1, 1);
  y1 ~ binomial(N1, theta1);
  y2 ~ binomial(N2, theta2);
}
generated quantities {
  real ve;
  ve = 1 - theta2 / theta1;
}
```

Vaccine efficacy

```
generated quantities {
  real ve;
  ve = 1 - theta2 / theta1;
}
```

- generated quantities is run after the sampling

Vaccine efficacy

```r
d_bin2 <- list(N1 = 338, y1 = 16, N2 = 867, y2 = 19)
mod_bin2 <- cmdstan_model(stan_file = 'binom2.stan')
fit_bin2 <- mod_bin2$sample(data = d_bin2, refresh=1000)
```

```
> Running MCMC with 4 parallel chains...

Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
...
All 4 chains finished successfully.
Mean chain execution time: 0.0 seconds.
Total execution time: 0.2 seconds.
```

Vaccine efficacy

```r
options(posterior.num_args=list(sigfig=2))
fit_bin2$summary()
```

```
  variable     mean   median     sd    mad       q5      q95  rhat ess_bulk ess_tail
  <chr>       <dbl>    <dbl>  <dbl>  <dbl>    <dbl>    <dbl> <dbl>    <dbl>    <dbl>
1 lp__     -164.    -163.    1.0    0.71   -166.    -163.      1.0    2014.    2483.
2 theta1      0.050    0.049 0.012  0.011     0.032    0.071   1.0    3468.    2714.
3 theta2      0.023    0.023 0.0050 0.0050    0.015    0.032   1.0    4000.    3061.
4 ve          0.51     0.54  0.17   0.15      0.21     0.73    1.0    3572.    2990.
```

Vaccine efficacy

```r
options(posterior.num_args=list(sigfig=2))
fit_bin2$summary()
```

```
  variable     mean   median     sd    mad       q5      q95  rhat ess_bulk ess_tail
  <chr>       <dbl>    <dbl>  <dbl>  <dbl>    <dbl>    <dbl> <dbl>    <dbl>    <dbl>
1 lp__     -164.    -163.    1.0    0.71   -166.    -163.      1.0    2014.    2483.
2 theta1      0.050    0.049 0.012  0.011     0.032    0.071   1.0    3468.    2714.
3 theta2      0.023    0.023 0.0050 0.0050    0.015    0.032   1.0    4000.    3061.
4 ve          0.51     0.54  0.17   0.15      0.21     0.73    1.0    3572.    2990.
```

- `lp__` is the log density, ie, same as `target`

HMC specific diagnostics

```
fit_bin2$diagnostic_summary(diagnostics = c("divergences",
                                            "treedepth"))
```

```
$num_divergent
[1] 0 0 0 0

$num_max_treedepth
[1] 0 0 0 0
```

HMC specific diagnostics

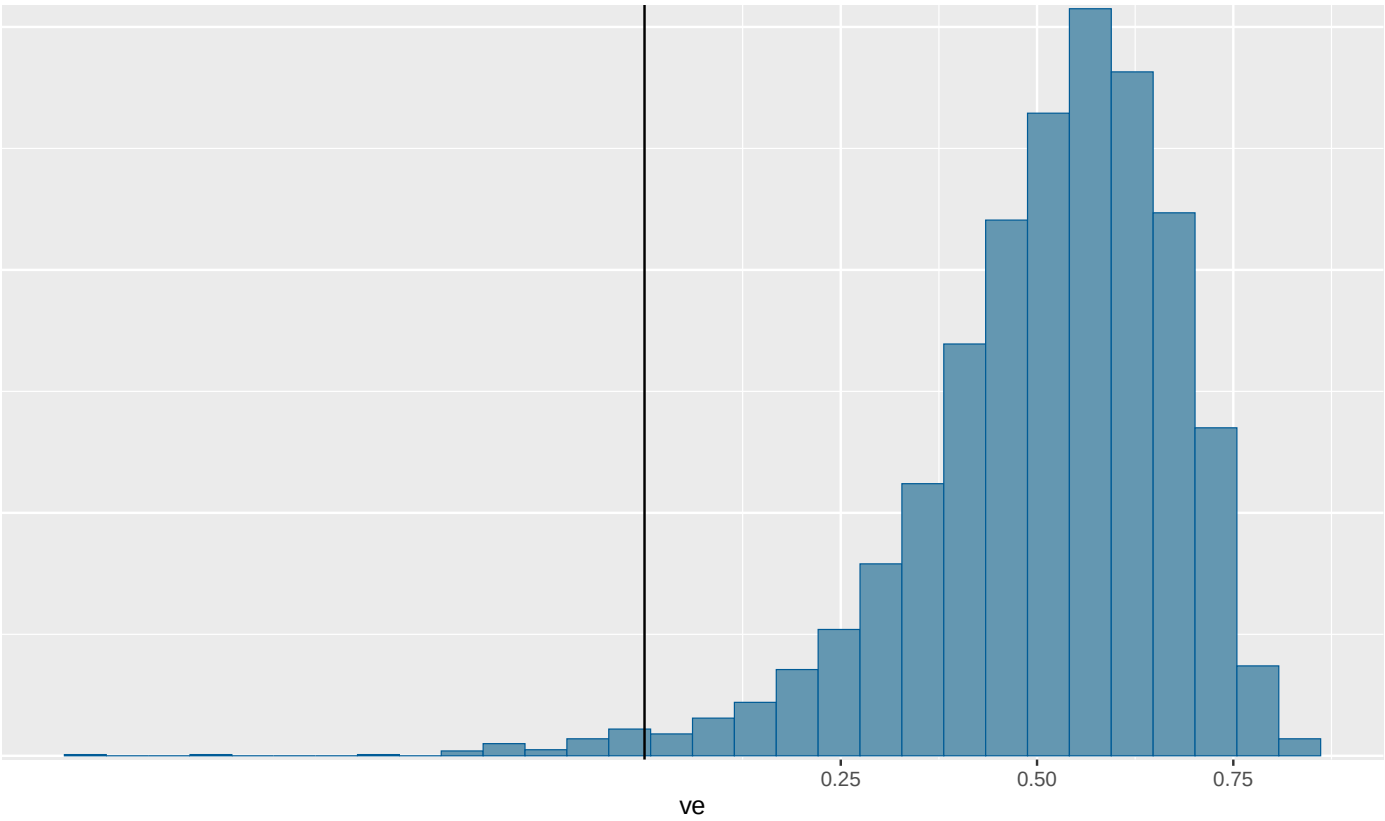
```
fit_bin2$diagnostic_summary(diagnostics = c("divergences",
                                            "treedepth"))
```

```
$num_divergent
[1] 0 0 0 0

$num_max_treedepth
[1] 0 0 0 0
```

`diagnostic_summary()` includes E-BFMI diagnostic, which I'll skip in this course

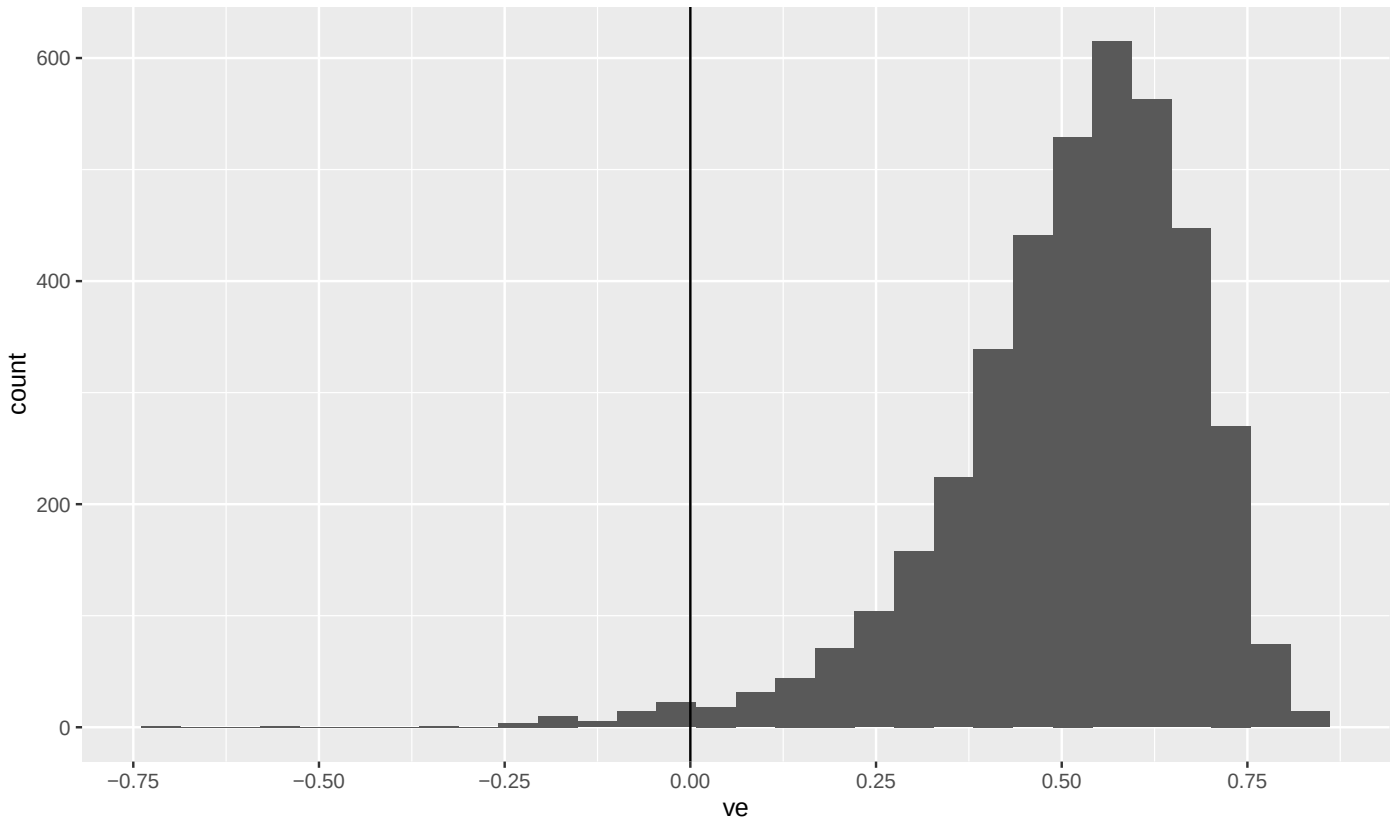
Vaccine efficacy (bayesplot)

```r
draws <- fit_bin2$draws(format = "df")
mcmc_hist(draws, pars = 've') +
  geom_vline(xintercept = 0) +
  scale_x_continuous(breaks = c(seq(-0.75,1,by=0.25)))
```



Vaccine efficacy (ggplot2)

```r
draws <- fit_bin2$draws(format = "df")
draws |> ggplot(aes(x=ve)) +
  geom_histogram() +
  geom_vline(xintercept = 0) +
  scale_x_continuous(breaks = c(seq(-0.75,1,by=0.25)))
```



Vaccine efficacy (probability and MCSE)

Probability (and corresponding MCSE) that VE<0

```
> draws |>
    mutate_variables(p_ve_lt_0 =
                       as.numeric(ve<0)) |>
    subset_draws("p_ve_lt_0") |>
    summarise_draws(prob=mean, MCSE=mcse_mean)

  variable   prob   MCSE
  p_ve_lt_0 0.013 0.0021
```

posterior object formats

Default is `draws_array`

```
> fit_bin2$draws()
# A draws_array: 1000 iterations, 4 chains, and 4 variables
, , variable = lp__

         chain
iteration    1    2    3    4
        1 -163 -164 -164 -163
        2 -163 -164 -165 -163
        3 -163 -166 -163 -163
        4 -164 -165 -164 -163
        5 -163 -167 -164 -164

, , variable = theta1

         chain
iteration     1     2     3     4
        1 0.061 0.048 0.068 0.055
        2 0.037 0.074 0.049 0.047
...
```

posterior object formats

`draws_df` looks prettier and works with `ggplot()`

```
> fit_bin2$draws(format ="df")
```

```
# A draws_df: 1000 iterations, 4 chains, and 4 variables
   lp__ theta1 theta2   ve
1  -163  0.061  0.025 0.59
2  -163  0.037  0.024 0.36
3  -163  0.044  0.020 0.55
4  -164  0.073  0.023 0.68
5  -163  0.040  0.022 0.46
6  -163  0.062  0.023 0.62
7  -165  0.072  0.018 0.75
8  -166  0.076  0.017 0.78
9  -165  0.069  0.017 0.76
10 -164  0.035  0.027 0.21
# ... with 3990 more draws
# ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

posterior object formats

`draws_rvar` makes it easy to compute derived quantities

```
> as_draws_rvars(fit_bin2$draws())
```

```
# A draws_rvars: 1000 iterations, 4 chains, and 4 variables
$lp__: rvar<1000,4>[1] mean ± sd:
[1] -164 ± 1

$theta1: rvar<1000,4>[1] mean ± sd:
[1] 0.05 ± 0.012

$theta2: rvar<1000,4>[1] mean ± sd:
[1] 0.023 ± 0.005

$ve: rvar<1000,4>[1] mean ± sd:
[1] 0.51 ± 0.17
```

posterior object formats

`draws_rvar` makes it easy to compute derived quantities

```
> as_draws_rvars(fit_bin2$draws())
# A draws_rvars: 1000 iterations, 4 chains, and 4 variables
$lp__: rvar<1000,4>[1] mean ± sd:
[1] -164 ± 1

$theta1: rvar<1000,4>[1] mean ± sd:
[1] 0.05 ± 0.012

$theta2: rvar<1000,4>[1] mean ± sd:
[1] 0.023 ± 0.005

$ve: rvar<1000,4>[1] mean ± sd:
[1] 0.51 ± 0.17
```

```
> with(as_draws_rvars(draws), 1 - theta2 / theta1)
rvar<1000,4>[1] mean ± sd:
[1] 0.51 ± 0.17
```

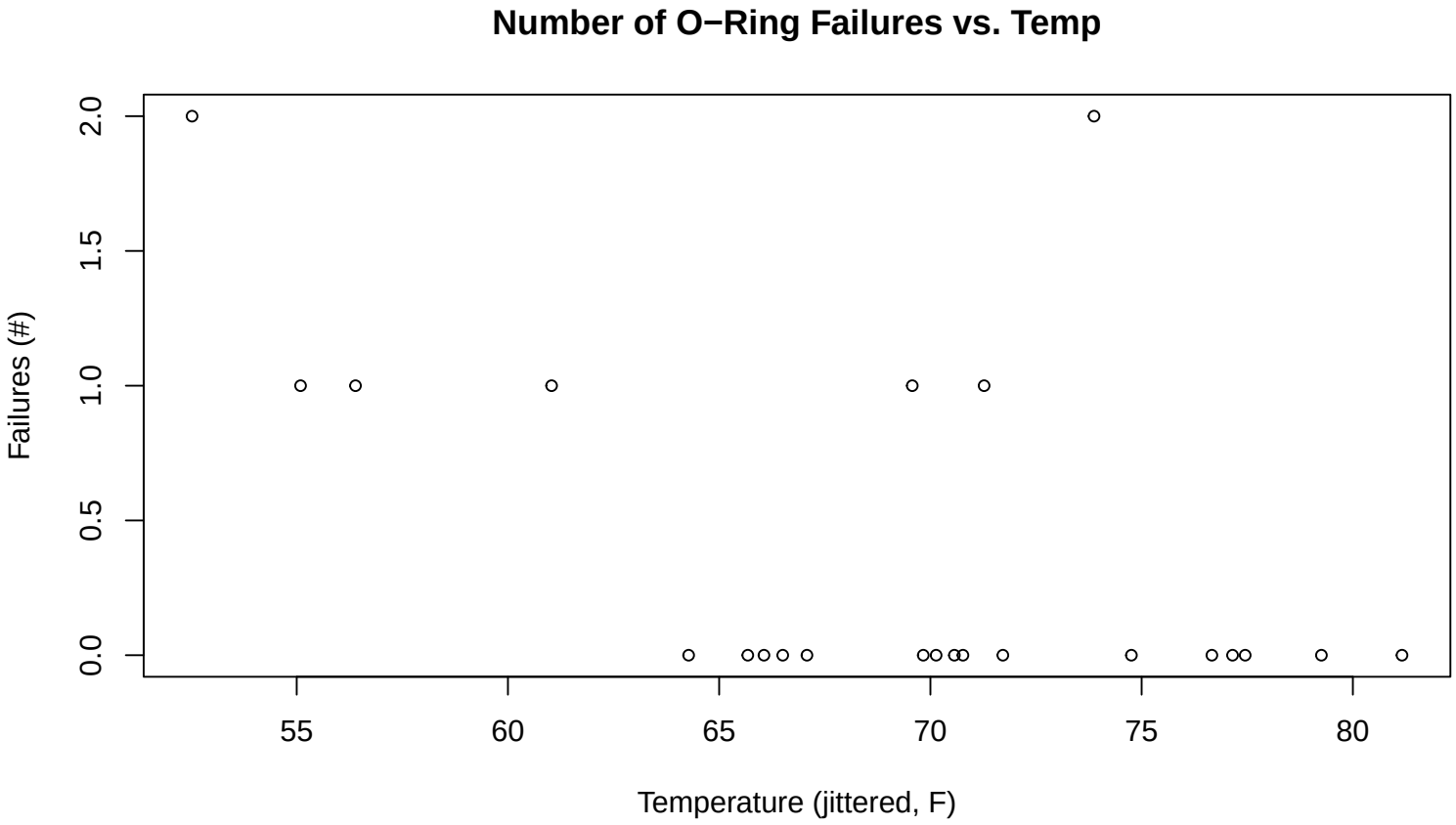
posterior object formats

`draws_rvar` makes it easy to compute derived quantities

```
> as_draws_rvars(fit_bin2$draws())
# A draws_rvars: 1000 iterations, 4 chains, and 4 variables
$lp__: rvar<1000,4>[1] mean ± sd:
[1] -164 ± 1

$theta1: rvar<1000,4>[1] mean ± sd:
[1] 0.05 ± 0.012

$theta2: rvar<1000,4>[1] mean ± sd:
[1] 0.023 ± 0.005

$ve: rvar<1000,4>[1] mean ± sd:
[1] 0.51 ± 0.17

> with(as_draws_rvars(draws), 1 - theta2 / theta1)
rvar<1000,4>[1] mean ± sd:
[1] 0.51 ± 0.17

> as_draws_rvars(draws)$ve<0
rvar<1000,4>[1] mean ± sd:
[1] 0.013 ± 0.11
```

Challenger dataset

- Number of failed o-rings from 23 Challenger launches vs. temperature
- Does temperature predict failure?



Logistic regression model

```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
}
parameters {
    real alpha;             // intercept
    real beta;              // slope
}
transformed parameters {
    vector[N] logit_p;
    logit_p = alpha + beta * x;  // logit-linear model
}
model {
    y ~ binomial_logit(K, logit_p); // observation model
}
```

Logistic regression model

```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
}
parameters {
    real alpha;             // intercept
    real beta;              // slope
}
transformed parameters {
    vector[N] logit_p;
    logit_p = alpha + beta * x;  // logit-linear model
}
model {
    y ~ binomial_logit(K, logit_p); // observation model
}
```

Logistic regression model

```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
}
parameters {
    real alpha;             // intercept
    real beta;              // slope
}
transformed parameters {
    vector[N] logit_p;
    logit_p = alpha + beta * x;  // logit-linear model
}
model {
    y ~ binomial_logit(K, logit_p); // observation model
}
```

Logistic regression model

```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
}
parameters {
    real alpha;             // intercept
    real beta;              // slope
}
transformed parameters {
    vector[N] logit_p;
    logit_p = alpha + beta * x;  // logit-linear model
}
model {
    y ~ binomial_logit(K, logit_p); // observation model
}
```

Logistic regression model

```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
}
```

- difference between `vector[N] x` and `array[N] real x`

Logistic regression model

```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
}
```

- difference between `vector[N] x` and `array[N] real x`
- only integer arrays: `array[N] int x`

Logistic regression model

```
parameters {
    real alpha;             // intercept
    real beta;              // slope
}
transformed parameters {
    vector[N] logit_p;
    logit_p = alpha + beta*x;  // logit-linear model
}
```

- transformed parameters are deterministic transformations of parameters and data

Robit model

```
...
parameters {
  real alpha;
  real beta;
  real<lower=1,upper=80> nu;
}
transformed parameters {
  vector[N] p;
  {
    vector[N] inv_student_t_cdf = alpha + beta*x;
    for (n in 1:N)
      p = student_t_cdf(inv_student_t_cdf, nu, 0, 1);
  }
}
model {
  nu ~ gamma(2, 0.1);              // prior for nu
  y ~ binomial(K, p);  // observation model
}
```

Priors for logistic regression

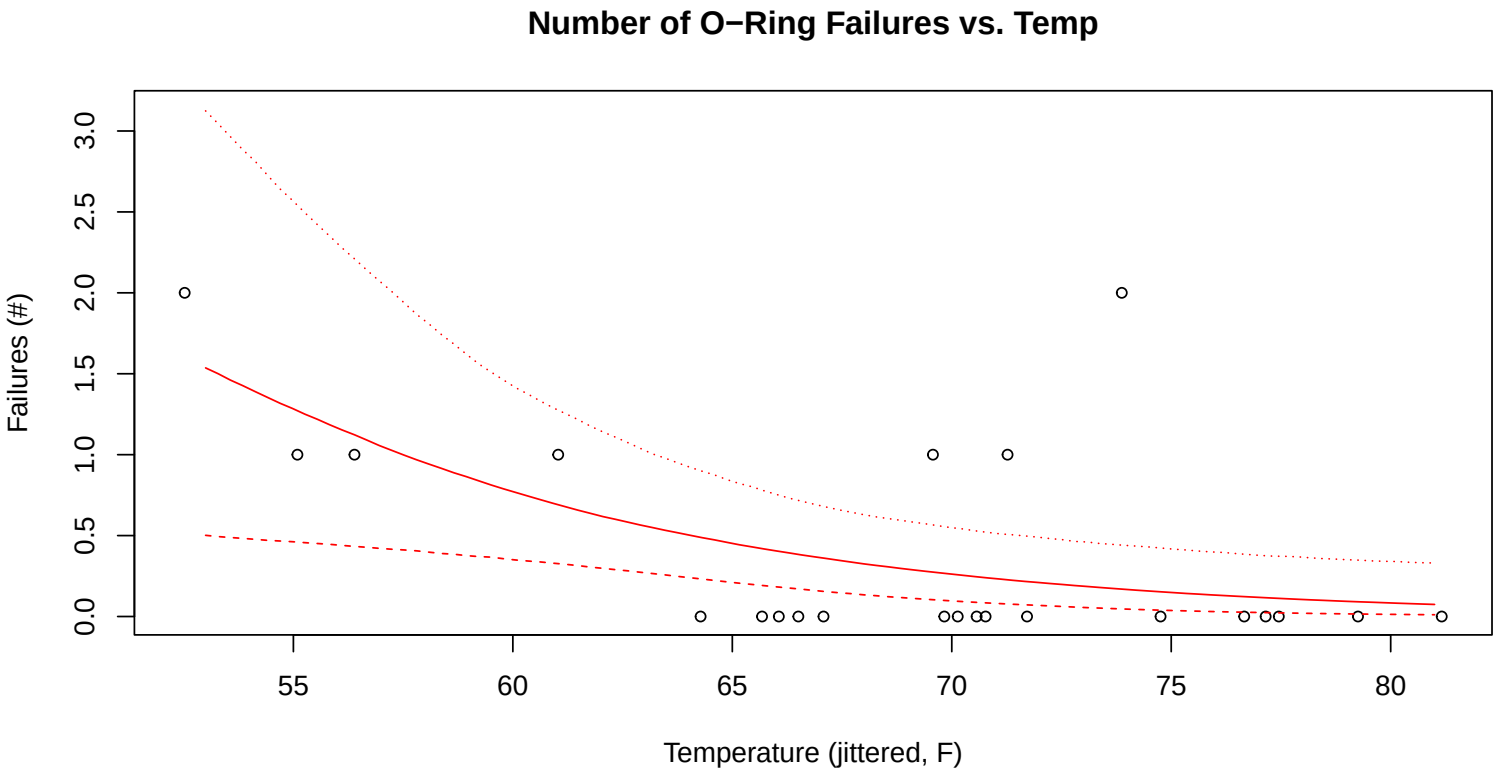
```
data {
    int<lower=0> N;         // number of observations
    vector[N] x;
    array[N] int<lower=1> K;
    array[N] int<lower=0> y;
    real pmualpha; // prior mean for alpha
    real psalpha;  // prior std for alpha
    real pmubeta;  // prior mean for beta
    real psbeta;   // prior std for beta
}
...
transformed parameters {
    vector[N] logit_p;
    logit_p = alpha + beta*x;
}
model {
    alpha ~ normal(pmualpha, psalpha); // prior for alpha
    beta ~ normal(pmubeta, psbeta);    // prior for beta
    y ~ binomial_logit(K, logit_p);             // observation mode
}
```

Priors

- Prior for temperature effect?

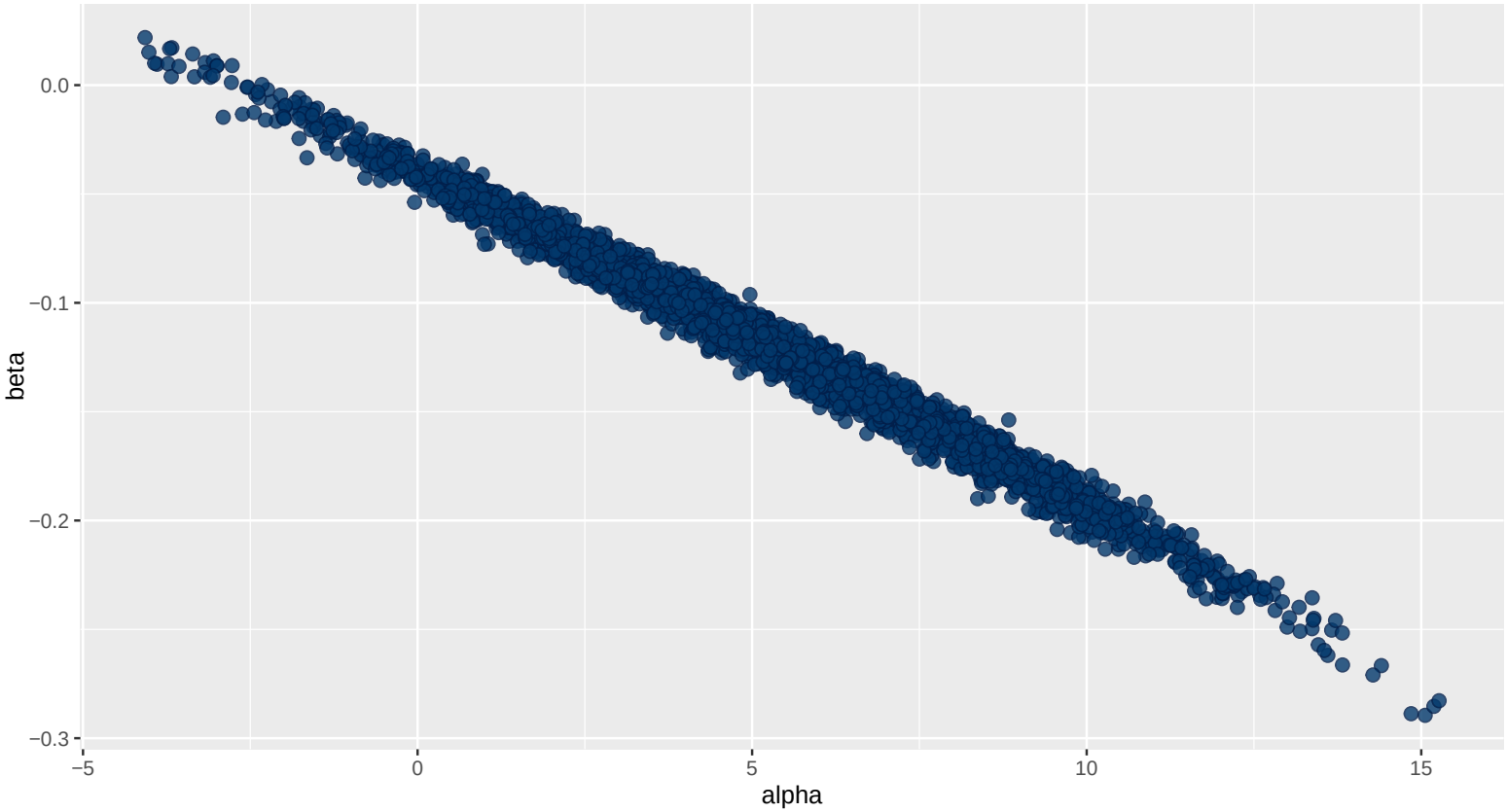
Challenger fit

Posterior fit



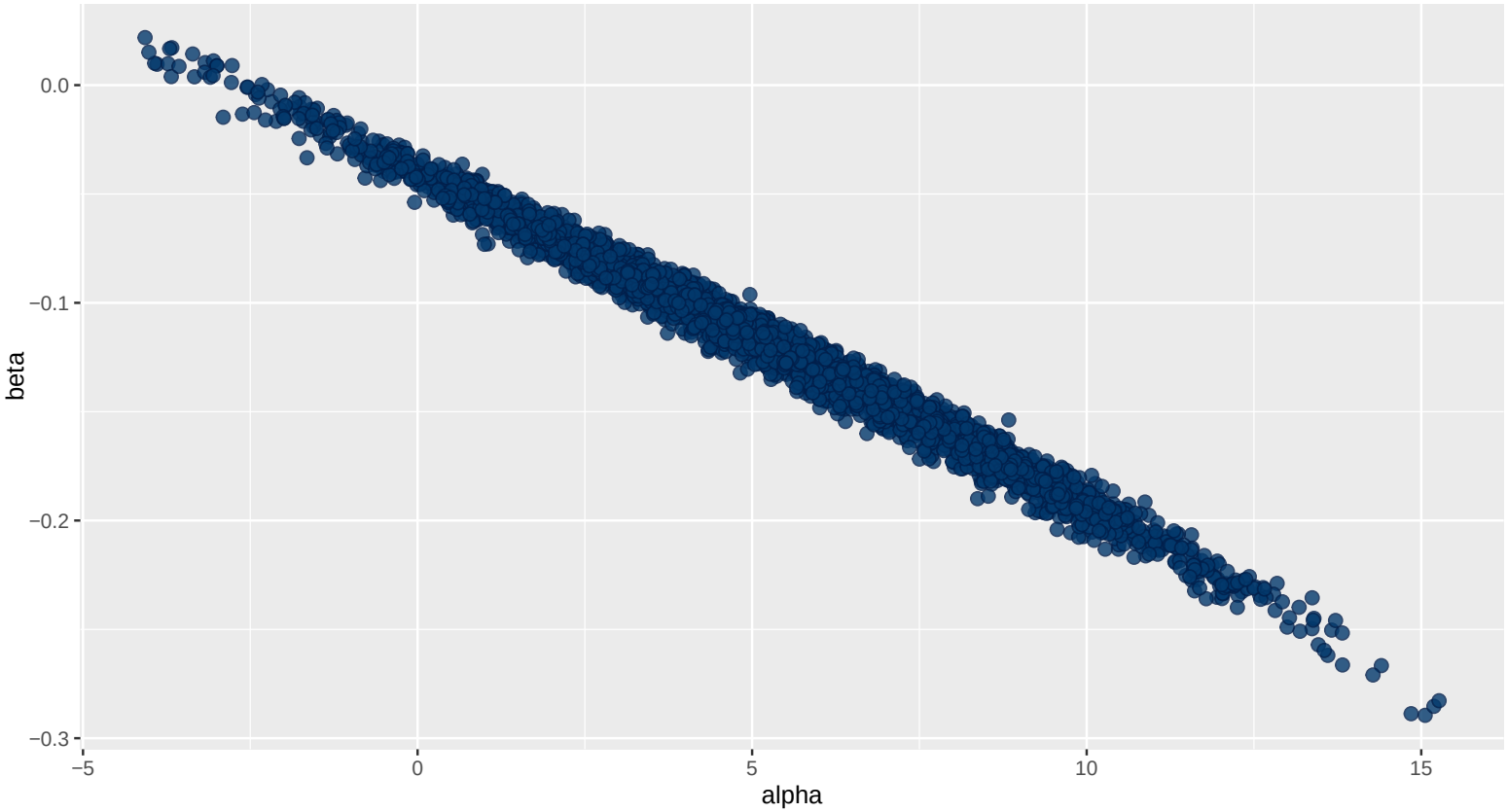
Challenger fit

Posterior draws of alpha and beta



Challenger fit

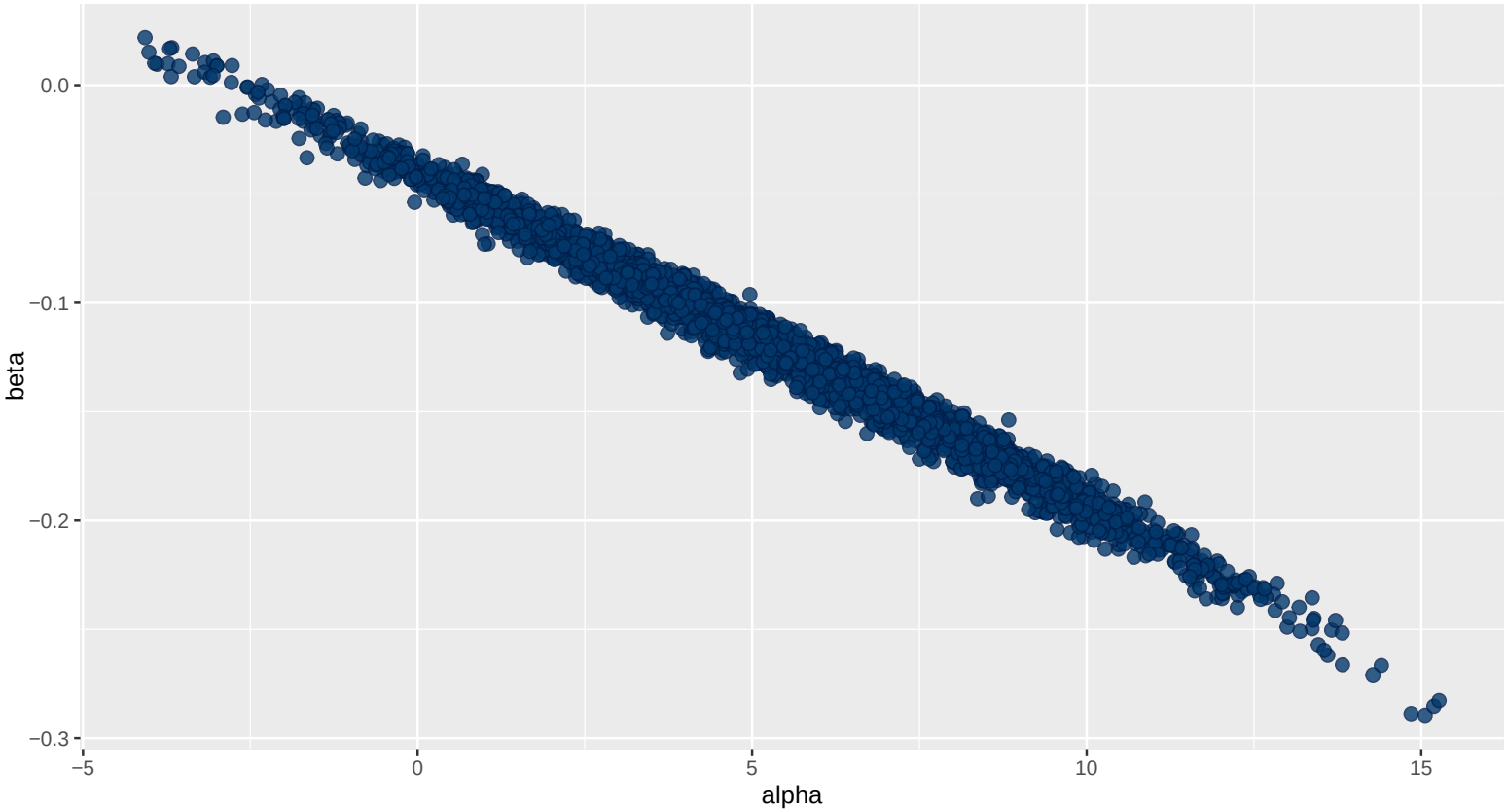
Posterior draws of alpha and beta




```
# A tibble: 3 × 10
  variable   mean median    sd   mad     q5     q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 lp__     -31.   -31.   0.96  0.75  -33.   -30.      1.0     955.    1282.
2 alpha      5.2    5.2  3.1   3.0    0.21  10        1.0     571      606
```

Challenger fit

Posterior draws of alpha and beta




```
# A tibble: 3 × 10
  variable   mean median    sd   mad     q5     q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 lp__     -31.   -31.   0.96  0.75  -33.   -30.      1.0     955.    1282.
2 alpha      5.2    5.2  3.1   3.0    0.21  10        1.0     571      606
```

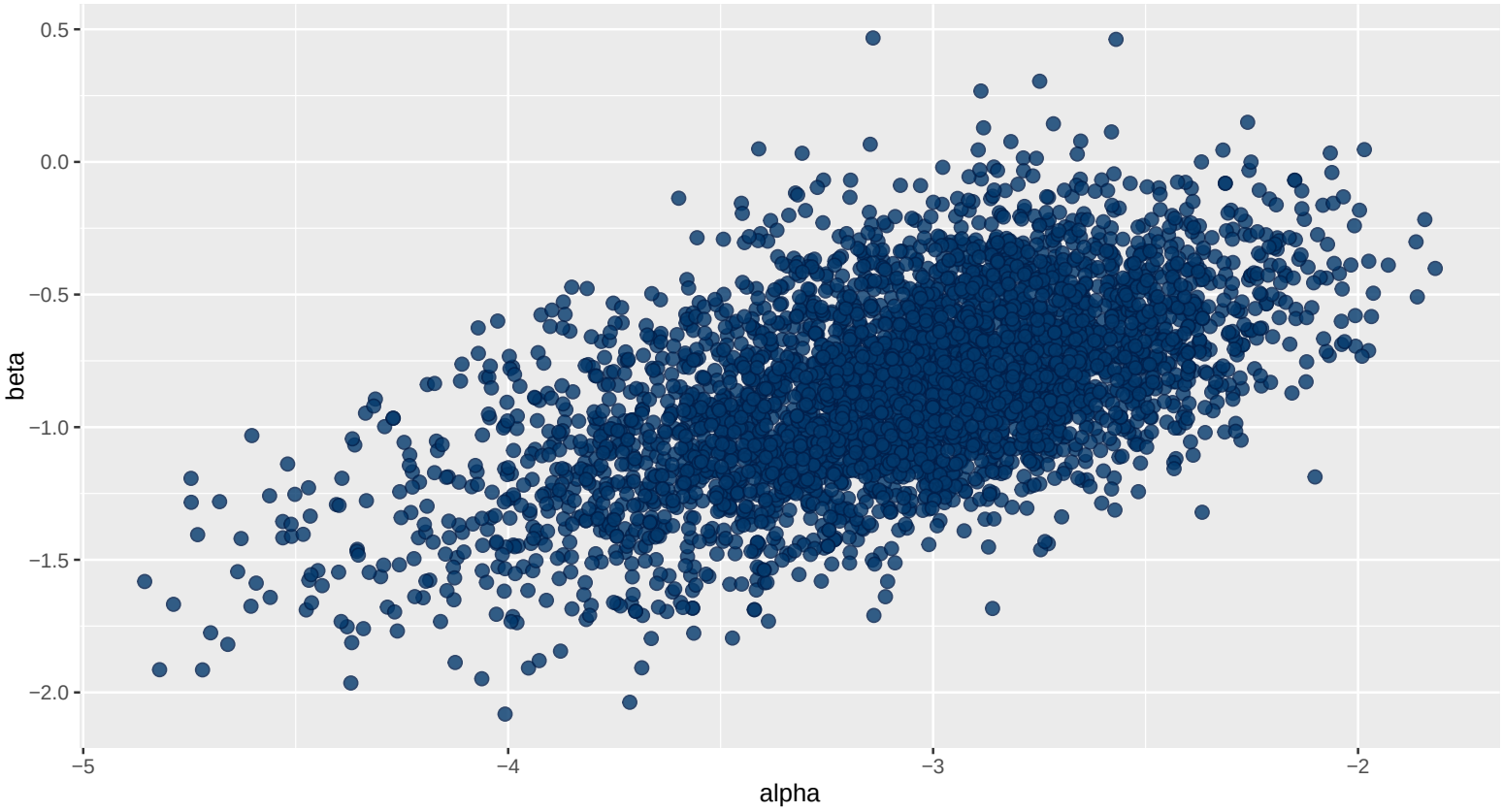
Logistic regression model

Center the data inside the model code

```
data {
  int<lower=0> N;        // number of observations
  vector[N] x;
  array[N] int<lower=1> K;
  array[N] int<lower=0> y;
  real xpred; // covariate value for prediction
}

transformed data {
  vector[N] x_std;
  real xpred_std;
  x_std = (x - mean(x)) / sd(x);
  xpred_std = (xpred - mean(x)) / sd(x);
}
```

Challenger fit

Posterior draws of alpha and beta when data is centered



Kilpisjärvi summer temperature

Without centering

```
> fit$summary(variables=c("alpha","beta"),
                  default_convergence_measures())
```

```
 variable  rhat ess_bulk ess_tail
 alpha      1.0     581.     607.
 beta       1.0     587.     650.
```

With centering

```
> fit_std$summary(variables=c("alpha","beta"),
                      default_convergence_measures())
```

```
 variable  rhat ess_bulk ess_tail
 alpha      1.0    1660.    1818.
 beta       1.0    1568.    1739.
```

RStanARM

- `RStanARM` provides simplified model description with pre-compiled models
  - no need to wait for compilation
  - a restricted set of models

Two group Binomial model:

```
d_bin2 <- data.frame(N = c(338, 867), y = c(16,19), grp2 = c(0,1))
fit_bin2 <- stan_glm(y/N ~ grp2,
                     weights = N,
                     family = binomial(),
                     data = d_bin2)
```

RStanARM

- `RStanARM` provides simplified model description with pre-compiled models
  - no need to wait for compilation
  - a restricted set of models

Two group Binomial model:

```
d_bin2 <- data.frame(N = c(338, 867), y = c(16,19), grp2 = c(0,1))
fit_bin2 <- stan_glm(y/N ~ grp2,
                     weights = N,
                     family = binomial(),
                     data = d_bin2)
```

Normal linear model

```
fit_lin <- stan_glm(fail ~ temp,
                    data = chl)
```

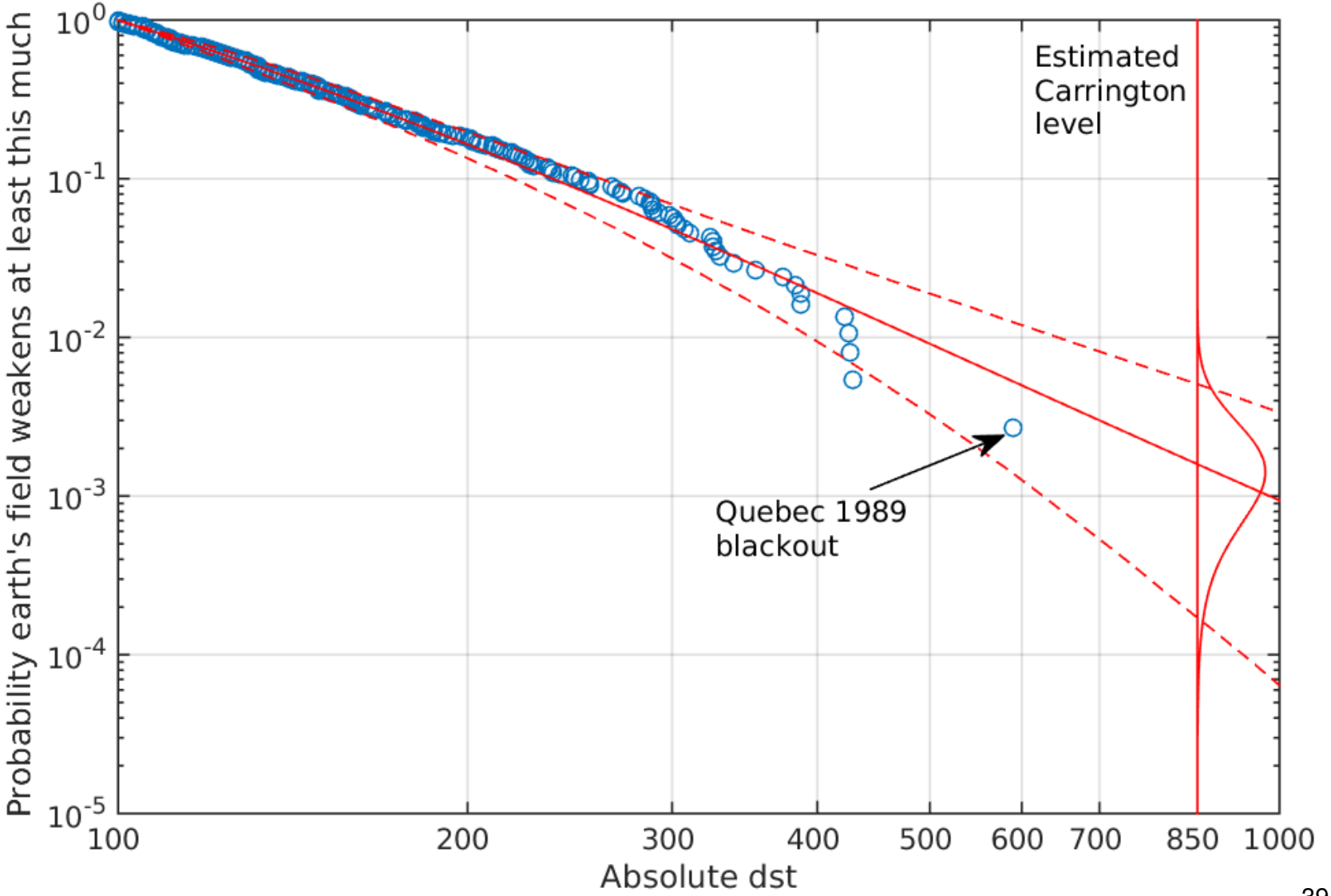
brms

- `brms` provides simplified model description
  - \+ a larger set of models than `RStanARM`, but still restricted
  - \- need to wait for the compilation

```
fit_bin2 <- brm(y | trials(N) ~ grp2,
                family = binomial(),
                data = d_bin2)

fit_lin_t <- brm(fail ~ temp,
                 family = student(),
                 data = d_lin)
```

Extreme value analysis

Geomagnetic storms



Extreme value analysis

```
data {
  int<lower=0> N;
  vector<lower=0>[N] y;
  int<lower=0> Nt;
  vector<lower=0>[Nt] yt;
}
transformed data {
  real ymax = max(y);        // pre-compute a useful quantity
}
parameters {
  real<lower=0> sigma;
  real<lower=-sigma/ymax> k; // constraint can depend on other parameters
}
model {
  y ~ gpareto(k, sigma);     // user defined distribution
}
generated quantities {
  vector[Nt] predccdf = gpareto_ccdf(yt, k, sigma);
}
```

Extreme value analysis

```
data {
  int<lower=0> N;
  vector<lower=0>[N] y;
  int<lower=0> Nt;
  vector<lower=0>[Nt] yt;
}
transformed data {
  real ymax = max(y);        // pre-compute a useful quantity
}
parameters {
  real<lower=0> sigma;
  real<lower=-sigma/ymax> k; // constraint can depend on other parameters
}
model {
  y ~ gpareto(k, sigma);     // user defined distribution
}
generated quantities {
  vector[Nt] predccdf = gpareto_ccdf(yt, k, sigma);
}
```

Extreme value analysis

```
data {
  int<lower=0> N;
  vector<lower=0>[N] y;
  int<lower=0> Nt;
  vector<lower=0>[Nt] yt;
}
transformed data {
  real ymax = max(y);        // pre-compute a useful quantity
}
parameters {
  real<lower=0> sigma;
  real<lower=-sigma/ymax> k; // constraint can depend on other parameters
}
model {
  y ~ gpareto(k, sigma);     // user defined distribution
}
generated quantities {
  vector[Nt] predccdf = gpareto_ccdf(yt, k, sigma);
}
```

Extreme value analysis

```
data {
  int<lower=0> N;
  vector<lower=0>[N] y;
  int<lower=0> Nt;
  vector<lower=0>[Nt] yt;
}
transformed data {
  real ymax = max(y);        // pre-compute a useful quantity
}
parameters {
  real<lower=0> sigma;
  real<lower=-sigma/ymax> k; // constraint can depend on other parameters
}
model {
  y ~ gpareto(k, sigma);     // user defined distribution
}
generated quantities {
  vector[Nt] predccdf = gpareto_ccdf(yt, k, sigma);
}
```

User defined functions

```
functions {
  real gpareto_lpdf(vector y, real k, real sigma) {
    // generalised Pareto log pdf with mu=0
    // should check and give error if k<0
    // and max(y)/sigma > -1/k
    int N;
    N <- dims(y)[1];
    if (abs(k) > 1e-15)
      return -(1+1/k)*sum(log1pv(y*k/sigma)) -N*log(sigma);
    else
      return -sum(y/sigma) -N*log(sigma); // limit k->0
  }
  vector gpareto_ccdf(vector y, real k, real sigma) {
    // generalised Pareto log ccdf with mu=0
    // should check and give error if k<0
    // and max(y)/sigma < -1/k
    if (abs(k) > 1e-15)
      return exp((-1/k)*log1pv(y/sigma*k));
    else
      return exp(-y/sigma); // limit k->0
  }
}
```

User defined functions

```
functions {
  real gpareto_lpdf(vector y, real k, real sigma) {
    // generalised Pareto log pdf with mu=0
    // should check and give error if k<0
    // and max(y)/sigma > -1/k
    int N;
    N <- dims(y)[1];
    if (abs(k) > 1e-15)
      return -(1+1/k)*sum(log1pv(y*k/sigma)) -N*log(sigma);
    else
      return -sum(y/sigma) -N*log(sigma); // limit k->0
  }
  vector gpareto_ccdf(vector y, real k, real sigma) {
    // generalised Pareto log ccdf with mu=0
    // should check and give error if k<0
    // and max(y)/sigma < -1/k
    if (abs(k) > 1e-15)
      return exp((-1/k)*log1pv(y/sigma*k));
    else
      return exp(-y/sigma); // limit k->0
  }
}
```

Different interfaces

- `CmdStanR` / `CmdStanPy`
  - Interface on top of command-line program CmdStan
- `RStan` / `PyStan`
  - C++ functions of Stan are called directly from R / Python
  - Higher integration between R/Python and Stan, but maybe more difficult to install due to more requirements of compatible C++ compilers and libraries

Other packages

- R
  - `posterior` — posterior handling and diagnostics (Lectures 5 and 6)
  - `bayesplot` — visualization and model checking (Lectures 5, 6, and 8)
  - `tidybayes` and `ggdist` — more posterior and prediction visualization (Lecture 6)
  - `marginaleffects` — prediction and comparison visualization
  - `loo` — cross-validation model assessment and comparison (Lecture 9)
  - `projpred` — projection predictive variable selection (Lecture 12)
  - `priorsense` — prior and likelihood sensitivity diagnostics (Lecture 12)

- Python
  - `ArviZ` — visualization, and model checking and assessment